

**Aim:** Create a Blockchain using Python

**Tasks Performed:**

1. **Blockchain Implementation** – Implemented the blockchain structure with block creation, transactions, proof-of-work, hashing, and chain validation.
2. **Flask REST API** – Developed Flask API endpoints for adding transactions, mining blocks, retrieving the blockchain, and validating the chain.
3. **ngrok Tunnel** – Exposed the local Flask server through a public ngrok URL for external API access.
4. **Transaction Addition** – Added transactions such as Alice → Bob and Charlie → David using the `/add_transaction` API.
5. **Block Mining using Proof-of-Work (PoW)** – Mined pending transactions into new blocks by solving the Proof-of-Work puzzle and generating a valid proof.
6. **Blockchain Retrieval** – Used the `/get_chain` API to retrieve and verify the complete blockchain, including the genesis block and mined blocks.
7. **Blockchain Validation** – Used `/is_valid` to verify the integrity of the blockchain and confirm that all blocks and hashes are valid.
8. **Postman API Testing** – Tested the Flask REST API endpoints using Postman and successfully verified transaction addition, block mining, blockchain retrieval, and blockchain validation.

**Theory:**

Blockchain is a distributed and immutable digital ledger that records data in a sequence of cryptographically linked blocks. Each block contains important information such as a timestamp, transactions, a proof generated using the Proof-of-Work (PoW) mechanism, and the hash of the previous block. Linking each block to the previous one helps maintain the integrity of the chain and makes unauthorized modification of stored data difficult.

In this experiment, a basic blockchain model is implemented using Python. Transactions are first added to a pending transaction pool and are then included in a new block after successful mining using Proof-of-Work. The SHA-256 hashing algorithm is used to generate unique hashes for blocks, while the blockchain validation mechanism checks the correctness of previous-hash references and Proof-of-Work conditions.

The blockchain is exposed through a Flask-based REST API, allowing users to add transactions, mine blocks, retrieve the complete blockchain, and verify its validity.

**Code & Output:****1. Install & Import Dependencies**

```
!pip install flask==2.2.5 pyngrok
import datetime
import hashlib
import json
```

**2. Blockchain Class / Implementation**

```
class Blockchain:
```

```
    def __init__(self):
        self.chain = []
        self.transactions = []
```

```
    self.create_block(
        proof=1,
        previous_hash='@',
        transactions=[]
    )
```

```
    # -----
    # CREATE TRANSACTION
    # -----
```

```
    def create_transaction(self, sender,
        receiver, amount):
```

```
        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }
```

```
        self.transactions.append(transaction)
```

```
        return len(self.chain) + 1
```

```
    # -----
    # CREATE BLOCK
    # -----
```

```
    def create_block(self, proof,
        previous_hash, transactions):
```

```
        block = {
            'index': len(self.chain) + 1,
```

```
            'timestamp':
                str(datetime.datetime.now()),
            'transactions': transactions,
            'proof': proof,
            'previous_hash':
                previous_hash
        }
```

```
        self.chain.append(block)
```

```
        return block
```

```
    # -----
    # GET PREVIOUS BLOCK
    # -----
```

```
    def get_previous_block(self):
```

```
        return self.chain[-1]
```

```
    # -----
    # PROOF OF WORK
    # -----
```

```
    def proof_of_work(self,
        previous_proof):
```

```
        new_proof = 1
        check_proof = False
```

```
        while check_proof is False:
```

```
            hash_operation =
                hashlib.sha256(
                    str(
                        new_proof**2 -
                        previous_proof**2
                    ).encode()
                ).hexdigest()
```

```

        if hash_operation[:3] == '000':
            check_proof = True

        else:
            new_proof += 1

    return new_proof

# -----
# HASH
# -----
def hash(self, block):

    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()

    return hashlib.sha256(
        encoded_block
    ).hexdigest()

# -----
# CHECK BLOCKCHAIN VALIDITY
# -----
def is_chain_valid(self, chain):

    previous_block = chain[0]

    block_index = 1

```

```

    while block_index < len(chain):

        block = chain[block_index]

        # Check whether
        previous_hash is correct
        if block['previous_hash'] !=
        self.hash(previous_block):
            return False

        # Check Proof of Work
        previous_proof =
        previous_block['proof']
        proof = block['proof']

        hash_operation =
        hashlib.sha256(
            str(
                proof**2 -
                previous_proof**2
            ).encode()
        ).hexdigest()

        if hash_operation[:3] != '000':
            return False

        previous_block = block
        block_index += 1

    return True

```

### 3. Flask API Initialization

```

from flask import Flask, jsonify, request
import threading

app = Flask(__name__)

blockchain = Blockchain()

```

**4. Home Route**

```
@app.route('/', methods=['GET'])
def home():

    return jsonify({
        'message': 'Blockchain API is
running successfully!',
        'endpoints': {
            'add_transaction':
'/add_transaction [POST]',
```

```
        'mine_block': '/mine_block
[GET]',
        'get_chain': '/get_chain [GET]',
        'is_valid': '/is_valid [GET]'
    }
    ), 200
```

**5. Add Transaction API**

```
@app.route('/add_transaction',
methods=['POST'])
def add_transaction():

    data = request.get_json()

    sender = data.get('sender')
    receiver = data.get('receiver')
    amount = data.get('amount')

    if not sender or not receiver or
amount is None:

        return jsonify({
            'message': 'Missing transaction
details.'
        }), 400
```

```
        next_block =
blockchain.create_transaction(
            sender,
            receiver,
            amount
        )

        return jsonify({
            'message': 'Transaction added
successfully.',
            'sender': sender,
            'receiver': receiver,
            'amount': amount,
            'next_block': next_block
        }), 201
```

**6. Mine Block API**

```
@app.route('/mine_block',
methods=['GET'])
def mine_block_api():

    if not blockchain.transactions:

        return jsonify({
            'message': 'No transactions
available. Add transactions before
mining.'
        }), 400
```

```
        previous_block =
blockchain.get_previous_block()

        previous_proof =
previous_block['proof']

        proof = blockchain.proof_of_work(
            previous_proof
        )
```

```

        previous_hash = blockchain.hash(
            previous_block
        )

        transactions_to_mine =
        blockchain.transactions.copy()

        blockchain.transactions.clear()

        block = blockchain.create_block(
            proof,
            previous_hash,
            transactions_to_mine
        )

        response = {
            'message': 'Congratulations, you
            just mined a block!',
            'index': block['index'],
            'timestamp': block['timestamp'],
            'transactions':
            block['transactions'],
            'proof': block['proof'],
            'previous_hash':
            block['previous_hash']
        }

        return jsonify(response), 200

```

## 7. Get Blockchain API

```

@app.route('/get_chain', methods=['GET'])
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

    return jsonify(response), 200

```

## 8. Validate Blockchain API

```

@app.route('/is_valid',
methods=['GET'])
def is_valid():

    valid = blockchain.is_chain_valid(
        blockchain.chain
    )

    if valid:

        response = {
            'message': 'All good. The
            Blockchain is valid.'
        }

        return jsonify(response), 200
    else:

        response = {
            'message': 'Houston, we have a
            problem. The Blockchain is not valid.'
        }

        return jsonify(response), 200

```

## 9. Run Flask Server

```
def run_flask():

    app.run(
        host='0.0.0.0',
        port=5000,
        debug=False,
        use_reloader=False
    )

    flask_thread = threading.Thread(
        target=run_flask
    )

    flask_thread.start()
```

## 10. ngrok Public Tunnel

```
public_url = ngrok.connect(5000)
```

```
print("Public URL:")
print(public_url)
```

```
▶ public_url = ngrok.connect(5000)

print("Public URL:")
print(public_url)

... Public URL:
NgrokTunnel: "https://motive-epilogue-deport.ngrok-free.dev" -> "http://localhost:5000"
```

## 11. Add Transaction — Testing Code

```
import requests

url = public_url.public_url +
"/add_transaction"

transaction = {
    "sender": "Alice",
    "receiver": "Bob",
    "amount": 50
}

response = requests.post(
    url,
    json=transaction
)

print("Status Code:",
      response.status_code)
print("Response:")
print(response.json())
```

```
... INFO:werkzeug:127.0.0.1 - - [19/Aug/2026 06:03:45] "POST /add_transaction HTTP/1.1" 201 -
Status Code: 201
Response:
{'amount': 50, 'message': 'Transaction added successfully.', 'next_block': 2, 'receiver': 'Bob', 'sender': 'Alice'}
```

## 12. Mine Block — Testing Code

```
import requests

url = public_url.public_url + "/mine_block"

response = requests.get(url)

print("Status Code:", response.status_code)
print("Response:")
print(response.json())
```

```
*** INFO:werkzeug:127.0.0.1 - - [19/Aug/2026 06:05:11] "GET /mine_block HTTP/1.1" 200 -
Status Code: 200
Response:
{'index': 2, 'message': 'Congratulations, you just mined a block!', 'previous_hash': 'c1c754496b279888aef13bfb76adeb39462221c225c48495277130d6b5aae976',
```

## 13. Get Blockchain — Testing Code

```
import requests

url = public_url.public_url + "/get_chain"

response = requests.get(url)

print("Status Code:", response.status_code)
print("Response:")
print(response.json())
```

### Output:

```
Pretty-print
{"chain": [{"index": 1, "previous_hash": "@", "proof": 1, "timestamp": "2026-08-19 06:02:02.984741", "transactions": []}, {"index": 2, "previous_hash": "c1c754496b279888aef13bfb76adeb39462221c225c48495277130d6b5aae976", "proof": 533, "timestamp": "2026-08-19 06:05:11.302686", "transactions": [{"amount": 50, "receiver": "Bob", "sender": "Alice"}]}, {"index": 3, "previous_hash": "0f7da132a7513a447ec2b465959174e1d21db32924b48a5d8f9241bc7c4bc1ea", "proof": 11028, "timestamp": "2026-08-19 06:06:01.155295", "transactions": [{"amount": 50, "receiver": "Bob", "sender": "Alice"}]}], "length": 3}
```

```
INFO:werkzeug:127.0.0.1 - - [19/Aug/2026 06:06:14] "GET /get_chain HTTP/1.1" 200 -
```

Status Code: 200

Response:

```
{'chain': [{'index': 1, 'previous_hash': '@', 'proof': 1, 'timestamp': '2026-08-19
06:02:02.984741', 'transactions': []}, {'index': 2, 'previous_hash':
'c1c754496b279888aef13bfb76adeb39462221c225c48495277130d6b5aae976', 'proof': 533,
'timestamp': '2026-08-19 06:05:11.302686', 'transactions': [{'amount': 50, 'receiver': 'Bob',
'sender': 'Alice'}]}, {'index': 3, 'previous_hash':
'0f7da132a7513a447ec2b465959174e1d21db32924b48a5d8f9241bc7c4bc1ea', 'proof':
11028, 'timestamp': '2026-08-19 06:06:01.155295', 'transactions': [{'amount': 50, 'receiver':
'Bob', 'sender': 'Alice'}]}], 'length': 3}
```

## 14. Validate Blockchain — Testing Code

```
import requests
```

```
url = public_url.public_url + "/is_valid"
```

```
response = requests.get(url)
```

```
print("Status Code:", response.status_code)
```

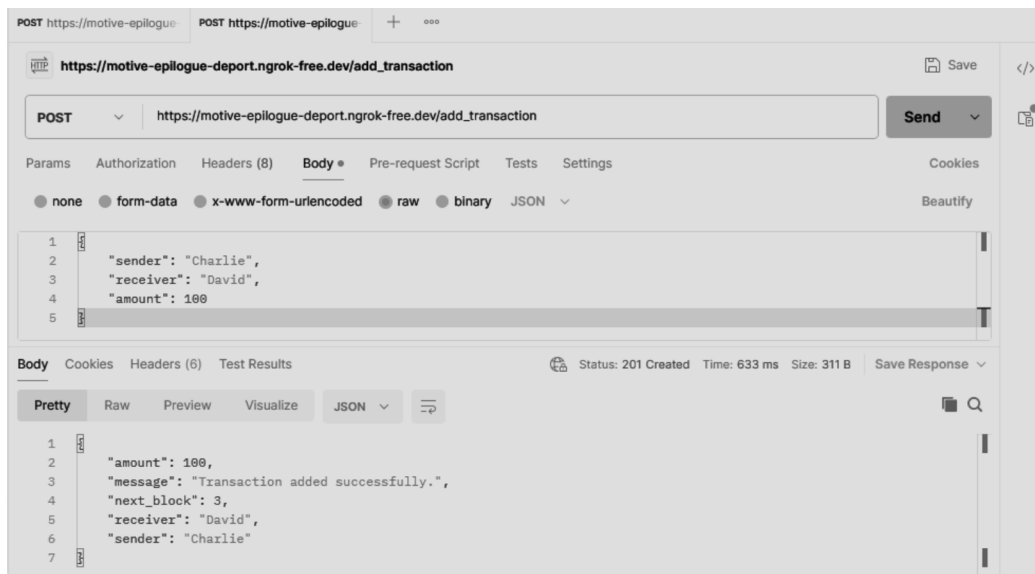
```
print("Response:")
```

```
print(response.json())
```

```
*** INFO:werkzeug:127.0.0.1 - - [19/Aug/2026 06:08:50] "GET /is_valid HTTP/1.1" 200 -  
Status Code: 200  
Response:  
{'message': 'All good. The Blockchain is valid.'}
```

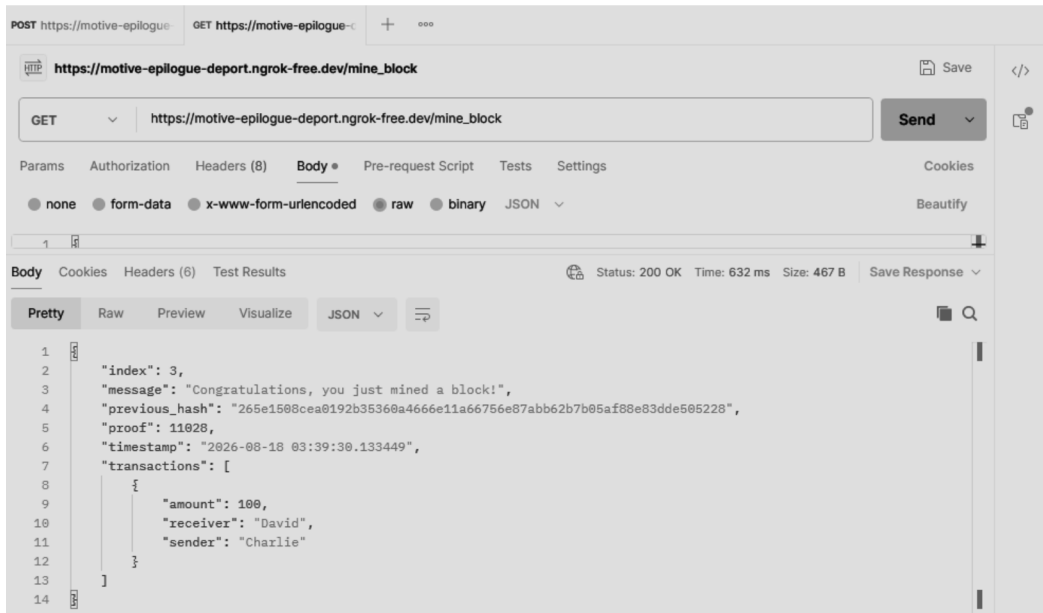
## Postman Commands & Output:

### Screenshot 1 — Add Transaction

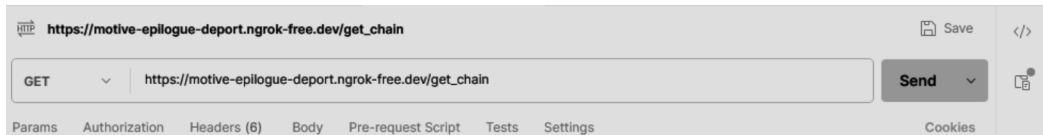




Screenshot 2 — Mine Block

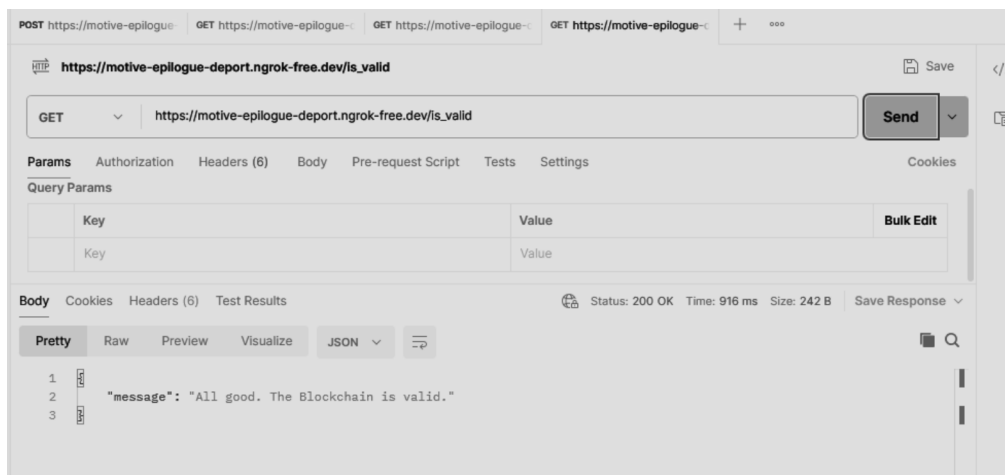


Screenshot 3 — Get Chain



```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "@",
      "proof": 1,
      "timestamp": "2026-08-18 03:27:14.304116",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "e6f69ef69575c133a30d9ce67fe78534b899c63670b3cdd0ad6bdd1b87c57d3a",
      "proof": 533,
      "timestamp": "2026-08-18 03:30:47.851102",
      "transactions": [
        {
          "amount": 50,
          "receiver": "Bob",
          "sender": "Alice"
        }
      ]
    }
  ]
}
```

## Screenshot 4 — Validate



## Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.